

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 03 -

ANÁLISE DE RISCOS: MITRE ATT&CK

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	10
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS.....	12

O QUE VEM POR AÍ?

A segurança não é um *feature* a ser adicionada, mas o alicerce fundamental do *software* moderno. Nesta aula, transformaremos a abordagem reativa em um processo proativo e automatizado, integrando o *DevSecOps* diretamente ao pipeline de entrega contínua.

O foco é duplo: proteger o código que a equipe de engenharia desenvolve (Análise Estática, SAST) e garantir a integridade de todos os componentes de terceiros herdados (Análise de Composição de Software, SCA). A meta é dominar a automação da segurança para que a conformidade regulatória se torne um subproduto natural e rastreável da sua arquitetura. Esteja preparado e preparada para operar o *shift left* na prática.

HANDS ON

A automação é a principal ferramenta do Arquiteto DevSecOps, e a prática valida a teoria. A jornada se inicia com a análise de segurança do código proprietário.

Implementaremos ferramentas de *Shift Left*, como o **Horusec**, diretamente no *workflow* de *build*, executando a Análise Estática de Segurança de Aplicações (SAST) e analisando os relatórios detalhados de vulnerabilidades de código.

Na etapa seguinte, o foco migrará para a Segurança da Cadeia de Suprimentos (SCA). Faremos o uso prático do *scanner open source* **Trivy** para varrer bibliotecas e dependências, configurando o pipeline para atuar como um *quality gate* que bloqueia o *deploy* automaticamente ao detectar CVEs críticas em componentes de terceiros. Concluiremos o ciclo de transparência com a geração automática do **Software Bill of Materials (SBOM)** no formato *CycloneDX*. Este é um artefato essencial para auditoria e conformidade, transformando os requisitos de *compliance* em código executável e rastreável.

SAIBA MAIS

O Imperativo do Shift Left: Análise Estática de Segurança de Aplicações (SAST)

A estratégia *Shift Left* é operacionalizada pelo Teste Estático de Segurança de Aplicações (SAST), um teste *white-box* que analisa o código-fonte (proprietário) para identificar falhas de codificação como Injeção de SQL ou *Cross-Site Scripting* (XSS) antes da execução. Ferramentas como o **Horusec** integram-se precocemente, fornecendo *feedback* imediato, seja no ambiente de desenvolvimento ou no *Pull Request*. A diferença essencial reside na metodologia: enquanto o SAST examina a lógica interna do código (estático), o DAST (*Dynamic Application Security Testing*) simula ataques externos em tempo de execução (teste *black-box*). A combinação de ambos endereça tanto problemas internos de codificação quanto a exposição externa.

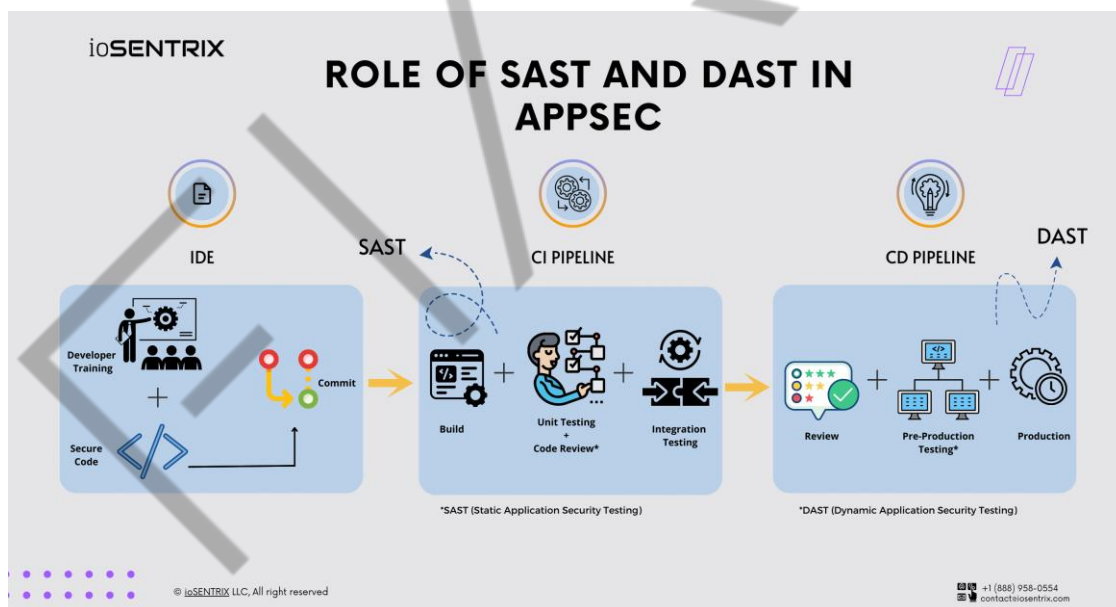


Figura 1 – SAST vs. DAST
Fonte: [CDN](#) (2025)

Um desafio inerente ao SAST é a geração de *False Positives*, onde o código seguro é erroneamente identificado como vulnerável. Para mitigar a *fadiga de alertas*, a maturidade em *DevSecOps* exige a calibração das regras da ferramenta. Uma abordagem avançada é utilizar o artefato VEX (*Vulnerability Exploitability*

Exchange) para complementar os achados de SAST, focando os esforços de remediação exclusivamente no risco real e mitigável. **Aproveite o momento e pense: como sua equipe está gerenciando a fadiga de alertas hoje?**

Segurança da Cadeia de Suprimentos (SCA) e a Gestão de Riscos de Terceiros

A Análise de Composição de Software (SCA) é crucial, visto que as aplicações modernas dependem predominantemente de código *open source*. O SCA varre a base de código para identificar componentes de terceiros e os compara com bancos de dados de vulnerabilidades conhecidas (CVEs), fornecendo visibilidade essencial para o gerenciamento de risco e licenças. Ferramentas como o **Trivy** permitem o monitoramento contínuo da cadeia de suprimentos. O risco mais sofisticado reside nas **dependências transitivas** (indiretas): pacotes exigidos por uma dependência direta, que introduzem vulnerabilidades invisíveis e difíceis de controlar, tornando-se alvos principais de ataques cibernéticos. Para mitigar esse vetor de ataque, as ferramentas de SCA devem mapear a árvore de dependências completa. A estratégia ideal é atualizar a dependência direta correspondente para uma versão que não utilize o componente indireto vulnerável, resolvendo o problema na origem e evitando o acúmulo de dívida de segurança oculta.



Figura 2 – SCA
Fonte: [Jetbrains](#) (2025)

Governança de Open Source e Compliance Legal de Licenças

A adoção do *open source* requer uma **Governança de Open Source** robusta para equilibrar agilidade e mitigação de riscos, garantindo a seleção adequada de bibliotecas e a aderência a requisitos regulatórios. O SCA é o mecanismo de detecção primário para o risco de licenciamento. As licenças se dividem em **Permissivas** (ex: MIT, Apache 2.0 etc.), que permitem uso em produtos proprietários com atribuição de crédito, e **Copyleft** (ex: GPL), que são restritivas e exigem que trabalhos derivados sejam distribuídos sob a mesma licença *open source* (risco de "viralidade"). Para *softwares* proprietários, a inclusão de componentes *Copyleft* fortes representa um risco estratégico, podendo forçar legalmente a abertura do código-fonte de todo o produto. Para prevenir isso, o DevSecOps Architect deve integrar a verificação de licenças do SCA para impor uma lista de permissão e bloquear automaticamente *builds* incompatíveis.

Para padronizar a comunicação transparente sobre licenças e a proveniência do *software*, o **Software Package Data Exchange (SPDX)** foi criado. Este padrão aberto, reconhecido como ISO/IEC 5962:2021, oferece um *framework* robusto, ideal para *compliance* de licenças e troca de metadados complexos, sendo suportado em múltiplos formatos como JSON e YAML.

Software Bill of Materials (SBOM): Transparência Automatizada

O **Software Bill of Materials (SBOM)** é um inventário formal e estruturado dos componentes que constituem um *software*, tornando-se um requisito de *due diligence* para avaliar o risco de segurança e conformidade legal. Sua confiabilidade depende da **geração automatizada** em cada *build* ou *release*, frequentemente via ferramentas de SCA como o Trivy ou *plugins* de CI/CD. Os formatos padronizados garantem a interoperabilidade: **CycloneDX** é leve e focado em segurança/BOV (*Bill of Vulnerabilities*), sendo preferido em *pipelines*, enquanto o **SPDX** é um padrão ISO/IEC, robusto e escolhido para conformidade legal e troca de dados complexos com fornecedores.

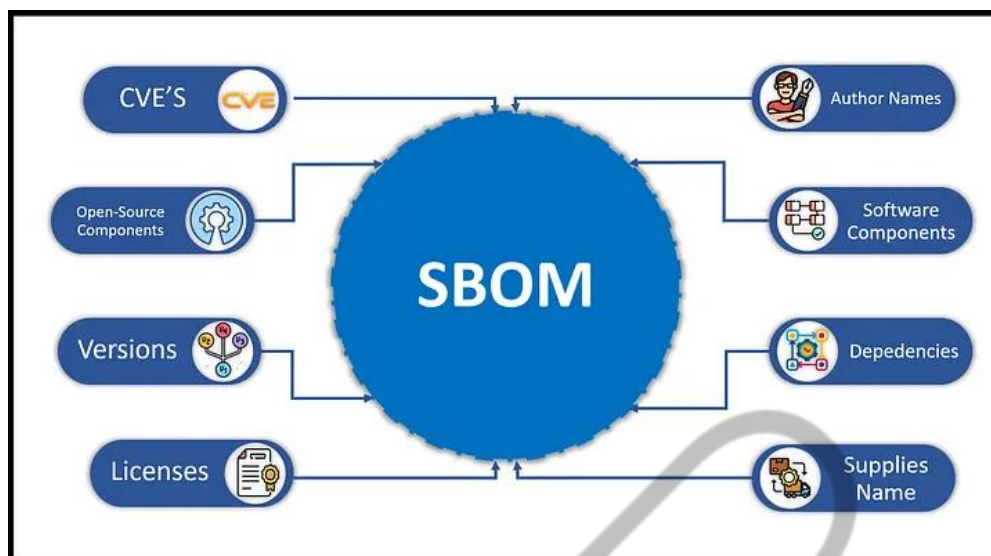


Figura 3 – SBOM
Fonte: [Medium](#) (2025)

O VEX (*Vulnerability Exploitability Exchange*) é um documento complementar que aborda o desafio da alta incidência de CVEs. O VEX reduz o "ruído" operacional ao declarar formalmente que uma CVE é **Não Explorável** no contexto do produto, permitindo que as equipes priorizem o *patching* apenas onde o risco é real e mitigável. O uso do VEX otimiza a alocação de recursos e melhora a postura geral de segurança.

Conformidade e Segurança da Cadeia de Suprimentos (NIST SSDF)

O *framework* **NIST SP 800-218: Secure Software Development Framework (SSDF)** é um conjunto de práticas sólidas para o desenvolvimento de *software* seguro, visando mitigar o risco de vulnerabilidades na cadeia de suprimentos. O SSDF exige a integração de práticas de segurança em todas as fases do *SDLC*, incluindo requisitos para fornecedores e garantindo a integridade do código, como a assinatura digital para evitar adulteração durante o processo de *build*.

A técnica principal para implementar *frameworks* como o NIST SSDF de forma escalável é o **Policy as Code (PaC)**, ou *Compliance as Code*. O PaC traduz especificações de conformidade e segurança em código declarativo, que é versionado no Git e automatizado nos *pipelines* de CI/CD, transformando requisitos regulatórios em *quality gates*. O motor de decisão central para o PaC é o **Open Policy Agent (OPA)**, que utiliza a linguagem Rego para expressar regras de segurança complexas, atuando como um ponto de decisão unificado em todo o

stack tecnológico (Kubernetes, CI/CD, etc.). Essa abordagem impõe a segurança de forma consistente e gera trilhas de auditoria abrangentes, essenciais para a prova de conformidade exigida por *frameworks* como o SSDF.

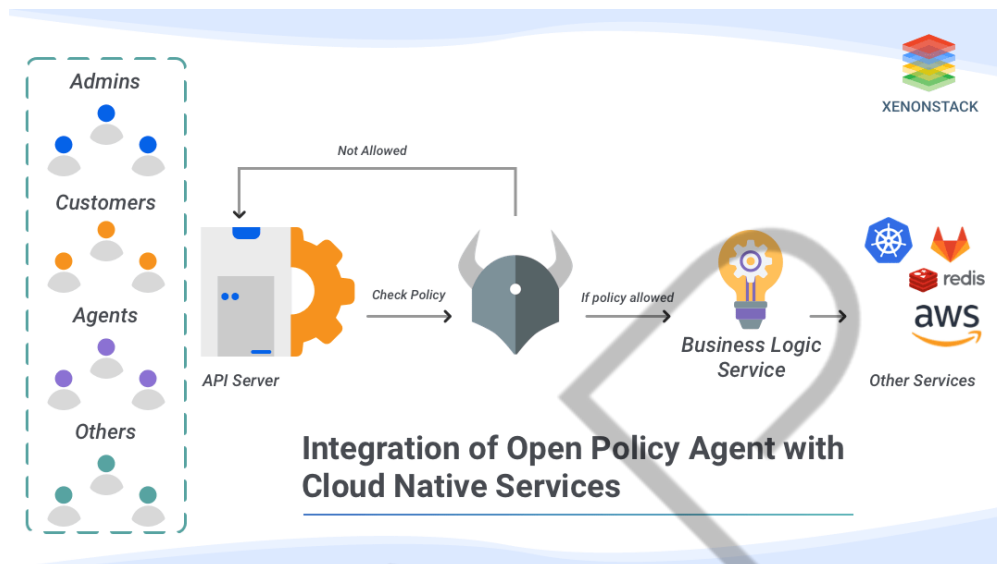


Figura 4 – Open Policy Agent (OPA)
Fonte: [Xenonstack](https://xenonstack.com) (2025)

MERCADO, CASES E TENDÊNCIAS

A integração estratégica de SAST, SCA e SBOM solidificou-se como um requisito de mercado. Esta abordagem é uma resposta direta à crescente complexidade dos ambientes *cloud-native* e à vulnerabilidade inerente da cadeia de suprimentos de *software*, sendo que incidentes de alto perfil que exploraram dependências de código aberto sublinharam a necessidade de controle e transparência absolutos.

A principal tendência que impulsiona a evolução do DevSecOps é a busca pela **priorização inteligente** do risco. O volume de alertas de segurança gerados por SAST e SCA é operacionalmente insustentável. A solução é a adoção do **Vulnerability Exploitability Exchange (VEX)**, que permite que as equipes filtrem o ruído e concentrem os limitados recursos de engenharia apenas nas vulnerabilidades que o VEX indica como exploráveis no contexto da aplicação. O VEX otimiza o tempo de resposta e garante que as correções sejam focadas onde o risco é real.

Adicionalmente, a convergência entre segurança e automação impulsionou a ascensão do **Policy as Code (PaC)**. À medida que os requisitos de *frameworks* de conformidade como o NIST SSDF são traduzidos para regras automatizadas (Rego no OPA), o DevSecOps move-se para um estado onde a conformidade é continuamente verificada e imposta. Esta governança codificada, alimentada pelos dados do SBOM/SCA, garante consistência e auditabilidade, acelerando a capacidade de entrega ao transformar as exigências regulatórias em código executável no *pipeline*. Essa sinergia é o futuro da arquitetura de *software* segura e ágil.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, expandimos a perspectiva sobre a integração de segurança nos pipelines de CI/CD. O conteúdo demonstrou como a estratégia *Shift Left* é operacionalizada pela Análise Estática (SAST) para proteger o código proprietário e pela Análise de Composição de Software (SCA) para mitigar o risco de dependências, incluindo a ameaça oculta das dependências transitivas.

Estabelecemos o papel central do *Software Bill of Materials (SBOM)* como artefato de transparência e auditoria, destacando os formatos padronizados CycloneDX e SPDX. Por fim, analisamos o imperativo da conformidade legal (licenças *Copyleft*) e a relevância de *frameworks* como o NIST SP 800-218 (SSDF), operacionalizados por meio do **Policy as Code** e do OPA, garantindo que as regras de segurança sejam aplicadas de forma automatizada e consistente.

REFERÊNCIAS

MCBRIDE, L. **Choosing the Right SBOM Standard: SPDX vs. CycloneDX**. 2023.

Disponível em: <https://www.sonatype.com/blog/comparing-sbom-standards-spdx-vs.-cyclonedx-vs.-swid> . Acesso em: 28 nov. 2025.

NIST. **Cybersecurity Framework 2.0: Quick-Start Guide for Cybersecurity Supply Chain Risk Management (C-SCRM)**. 2024. Disponível em:

<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.1305.pdf>. Acesso em: 28 nov. 2025.

OPEN POLICY AGENT. **Documentation**. 2025. Disponível em:

<https://openpolicyagent.org/>. Acesso em: 28 nov. 2025.

PETERSON, J. **How SCA and SAST Work Together for Security**. 2025. Disponível

em: <https://cyclo.com/blog/how-sca-and-sast-work-together-for-security>. Acesso em: 28 nov. 2025.

TRIVY. **SBOM scanning**. 2025. Disponível em:

<https://trivy.dev/docs/latest/target/sbom/>. Acesso em: 28 nov. 2025.

PALAVRAS-CHAVE

A SAST. SCA. DevSecOps. SBOM. CycloneDX. SPDX. Dependência Transitiva.
Compliance Legal. NIST SSDF. Policy as Code. Horusec. Trivy. VEX.

EMANIP



POS TECH